

CX 4220/CSE 6220 High Performance Computing
Midterm I Solutions

1. (4 points)

The best sequential and parallel algorithms for solving a problem have the following run-times:

$$T(n, 1) = \Theta(n)$$

$$T(n, p) = \Theta\left(\frac{n}{p} + \sqrt{p}\right)$$

What is the maximum number of processors (as a function of n) that can be used while still achieving optimal efficiency?

Answer:

$$\begin{aligned} E(p) &= \Theta(1) \\ \implies \frac{T(n, 1)}{pT(n, p)} &= \Theta(1) \\ \implies \frac{n}{p \cdot \left(\frac{n}{p} + \sqrt{p}\right)} &= \Theta(1) \\ \implies \frac{n}{n + p\sqrt{p}} &= \Theta(1) \\ \implies p\sqrt{p} &= O(n) \\ \implies p &= O(n^{2/3}) \end{aligned}$$

2. (5 points) The following algorithms are available for solving a problem of size n . A serial algorithm S that takes $\Theta(n^2)$ time, a parallel algorithm A_1 that takes $\Theta(n)$ time on n processors, and another parallel algorithm A_2 that takes $\Theta(\log n)$ time on n^2 processors.

- (a) Which parallel algorithm gives you maximum speed? (1 point)
- (b) Which parallel algorithm is more efficient? (2 points)
- (c) Which algorithm should be recommended for practical use? Is there a clear choice? (1 point)
- (d) Now, suppose A_1 consumes 100 bytes of memory per processor when run on n processors and A_2 consumes 10 bytes of memory per processor when run on n^2 processors. Taking this new information into account, which algorithm should be recommended? (1 point)

Answer:

$$T(n, 1) = \Theta(n^2); T_{A_1}(n, n) = \Theta(n); T_{A_2}(n, n^2) = \Theta(\log n)$$

- (a) A_2 gives the maximum speed since its runtime of $\Theta(\log n)$ is smaller.
- (b) Algorithm A_1 : $E_{A_1}(n, p) = \Theta(\frac{n^2}{n \times n}) = \Theta(1)$
 Algorithm A_2 : $E_{A_2}(n, p) = \Theta(\frac{n^2}{n^2 \times \log n}) = \Theta(\frac{1}{\log n})$
 A_1 is more efficient.
- (c) Assuming $p \leq n$, which is most often the case in practice, the more efficient algorithm A_1 is recommended. It gives optimal efficiency as well as maximum speed/speedup on p processors.
- (d) When run on $p \leq n$ processors, the memory requirement of A_1 is $\frac{100n}{p}$ and that of A_2 is $\frac{10n^2}{p}$. This further bolsters the case for using the more efficient algorithm in practice.
3. (5 points) Given n numbers $x_0, x_1, x_2, \dots, x_{n-1}$, we wish to compute $s_0, s_1, \dots, s_{n-1}$ defined as:

$$s_0 = x_0$$

$$s_i = \min(2s_{i-1}, x_i)$$

Design a parallel algorithm for solving this problem and compute its run-time.

Answer:

Define $y_i = \frac{s_i}{2^i} = \min(\frac{2s_{i-1}}{2^i}, \frac{x_i}{2^i}) = \min(\frac{s_{i-1}}{2^{i-1}}, \frac{x_i}{2^i}) = \min(y_{i-1}, \frac{x_i}{2^i})$.

Compute $y_0, y_1, \dots, y_{n-1}$ by computing parallel prefix on $x_0, \frac{x_1}{2}, \frac{x_2}{2^2}, \dots, \frac{x_{n-1}}{2^{n-1}}$ with $\min$ as the operation.

For each $0 \leq i < n$: $s_i = y_i \times 2^i$.

The run-time is dominated by parallel prefix.

Computation time: $O\left(\frac{n}{p} + \log p\right)$

Communication time: $O((\tau + \mu) \log p)$

4. (5 points) Consider n line segments $l_1, l_2, \dots, l_n$ in one dimension. Each line segment l_i is represented by two tuples (x_i, i, L) and (y_i, i, R) denoting its endpoints, where x_i is the left endpoint and y_i is the right endpoint of l_i . Assume the input is given as $2n$ endpoints in sorted order based on the first value of the tuples, and each endpoint is distinct from all others. Assuming such an input is available in block distributed fashion on p processors, design a parallel algorithm to compute the maximum number of line segments that overlap each other, and specify its run-time.

Answer:

Let L denote the input array of sorted endpoints of size $2n$.

Define array A of size $2n$ such that $A[i] = 1$ if $L[i]$ is a left end point, and $A[i] = -1$ if $L[i]$ is a right end point.

Compute parallel prefix sums on A and store the results in array PS .

Find the maximum number in PS using parallel reduction.

The run-time is dominated by parallel prefix.

Computation time: $O\left(\frac{n}{p} + \log p\right)$

Communication time: $O((\tau + \mu) \log p)$

5. (6 points) **Largest sum subarray:** Let A be an array of size n containing real numbers which can be positive or negative. Consider the problem of finding a consecutive subarray of A with the goal of maximizing the total sum of elements in the subarray. Formally, we wish to find $0 \leq i, j < n$ such that $\sum_{k=i}^j A[k]$ is maximized. Design a parallel algorithm for solving this problem and compute its run-time.

Answer: Let $A[a \cdots b]$ denote the subarray of A that starts at index a and ends at index b , and let $Sum(A[a \cdots b]) = \sum_{k=a}^b A[k]$. Let PS denote the array of prefix sums of A .

PS is defined in such a manner:

$$PS[-1] = 0$$

$$PS[i] = PS[i-1] + a[i] \text{ where } i \in [0, n)$$

For a fixed b , let a denote the index of the smallest value in $PS[-1 \dots b]$. Then, it follows that $A[a+1 \cdots b]$ is the largest sum subarray of A that ends at b .

Proof: Suppose not, and there exists $l \neq a+1$ such that $Sum(A[l \cdots j]) > Sum(A[i+1 \cdots j])$. But $Sum(A[i+1 \cdots j]) = PS[j] - PS[i] \geq PS[j] - PS[l-1] = Sum(A[l \cdots j])$ (since $PS[i]$ is the minimum, a contradiction).

Based on this, the following is the parallel algorithm for the largest sum subarray problem:

- Compute PS as defined above from A using parallel prefix sums.
- Run parallel prefix on PS with min as the binary associative operation and store results in min_PS .
- Compute, for each $0 \leq j < n$: $PS[j] - min_PS[j]$.
- Run Reduce on the above series of values with max as the operation to find the largest sum of a subarray.

Note that the above algorithm finds the largest sum of a subarray, but does not identify the subarray itself. To do so, create array of tuples $(PS[k], k)$ and run parallel prefix with min as the binary associative operation on the first element of the tuples. This will carry index k along with the minimum PS value found. Then, at the entry that maximizes $PS[j] - min_PS[j]$, both j and corresponding starting index $i = k + 1$ would be known.

Adding run-times of the individual steps together,

Computation time: $O\left(\frac{n}{p} + \log p\right)$

Communication time: $O((\tau + \mu) \log p)$

Example: In the example given below, maximum value of difference $[(3, 1)]$ is achieved at index 4, so $j = 4$, and $i = k + 1 = 2$.

idx	-1	0	1	2	3	4
A		1	-5	1	1	1
$(PS[k], k)$	(0, -1)	(1, 0)	(-4, 1)	(-3, 2)	(-2, 3)	(-1, 4)
min_PS		(0, -1)	(-4, 1)	(-4, 1)	(-4, 1)	(-4, 1)
$PS - min_PS$		(1, -1)	(0, 1)	(1, 1)	(2, 1)	(3, 1)

Table 1: Example for Q5